PharmaSafe: Product Architecture and Build Prompt

Oct 25, 2026 · @John

Scope and assumptions

PharmaSafe is a multi-tenant SaaS product that turns adverse event reports from any channel into validated, coded, regulator-ready safety cases. AI does the reading and drafting; trained safety staff make every regulated decision.

Target customers: mid-size pharma and biotech companies, and pharmacovigilance service providers that process cases for many clients.

Product-grade means four things beyond a portfolio build:

1. **Compliant:** it supports formal validation under GxP, 21 CFR Part 11 and EU Annex 11.
2. **Interoperable:** it uses E2B(R3) for case exchange, plus MedDRA and WHODrug coding.
3. **Reliable at scale:** it meets uptime, durability and deadline guarantees.
4. **Governed AI:** every model and prompt version is tested, approved, monitored and reversible.

Design assumptions (confirm with real customers):

Assumption	Value
Peak volume per large tenant	100,000 cases per month
Concurrent reviewers per tenant	Up to 300
AI processing time per case (p95)	Under 2 minutes
Availability	99.9% monthly
Data loss tolerance (RPO)	15 minutes
Recovery time (RTO)	4 hours
Record retention	Configurable per tenant; default 25 years
Primary cloud	Microsoft Azure (cloud-neutral design; AWS possible later)

The software described here is necessary but not sufficient. Licences, validation evidence, regulator gateway onboarding and quality processes are organisational work, covered in the last section.

Requirements

The product must cover the full case journey, from intake to regulator acknowledgment, with a human approval at every regulated step.

Functional

1. **Multi-channel intake:**
   - A monitored email inbox.
   - Web and call-centre forms.
   - Partner E2B(R3) files.
   - Scanned PDFs and faxes, read with OCR.
   - A literature feed from PubMed.
   - An API for customer systems.
2. **AI-assisted case processing:**
   - Personal-data redaction.
   - Structured extraction with evidence for each field.

- Validity checks against the 4 minimum criteria.
- Seriousness assessment.
- Duplicate detection.
- MedDRA and WHODrug coding suggestions.
- Follow-up questions.
- A narrative draft.

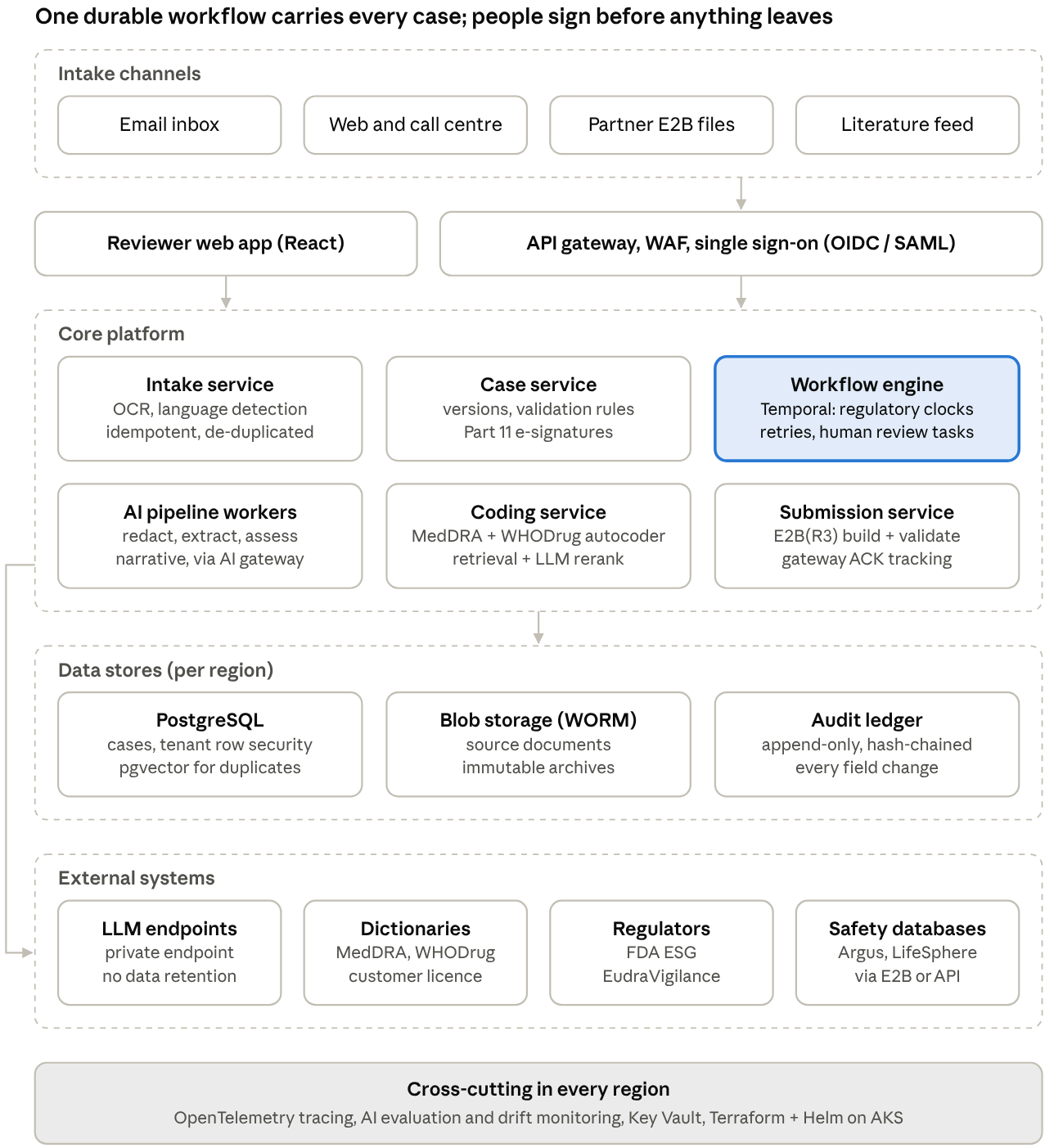
3. **Human workflow:**
   - Queues by tenant, product and region.
   - Assignment and peer-review (four-eyes) checks.
   - Medical review for serious cases.
   - Electronic signatures.
   - Follow-up versions of a case.
4. **Regulatory clocks:** day 0 is when the minimum criteria are first known. Due dates are set by configurable rule sets for each regulator, such as FDA, EMA and CDSCO. Escalations fire before a breach.
5. **Submission:**
   - Generate E2B(R3) XML and check it against ICH and regional business rules.
   - Send it through regulator gateways and track the acknowledgments.
   - Export to customer safety databases.
6. **Search and reporting:**
   - Case search.
   - Line listings.
   - On-time compliance and backlog dashboards.
   - An audit trail viewer.
7. **Administration:**
   - Tenant setup.
   - Single sign-on with the customer's identity provider.
   - Role and data-scope management.
   - Dictionary version loading.
   - AI automation settings per tenant.

Non-functional

Area	Requirement
Compliance	Part 11 and Annex 11 controls; supports GAMP 5 validation; tamper-evident audit trail
Privacy	GDPR, HIPAA and India's DPDP Act; data residency by region (EU, US, India)
Security	Encryption in transit and at rest; customer-managed keys option; least-privilege access; yearly penetration test
Availability	99.9% per month; no single point of failure in production
Durability	RPO 15 minutes, RTO 4 hours; source documents never lost once acknowledged
Performance	API p95 under 300 ms; AI processing p95 under 2 minutes per case; review screen loads in under 2 seconds
Correctness	No case is lost or processed twice; every state change is idempotent and traceable
AI quality	Seriousness recall at least 0.98 on the validation set before any release; quality monitored continuously
Auditability	Every case version can be rebuilt as it existed on any past date
Operability	Zero-downtime deploys; full traceability from a reviewer's action to logs, traces and model calls

Architecture

PharmaSafe is a modular platform deployed per region. A durable workflow engine owns each case from intake to submission, and every outside system is reached through one controlled service.



PharmaSafe architecture · one region

Reports enter through intake channels or the API gateway. The workflow engine calls the AI pipeline, coding and submission services in turn, and waits for signed human review between them. All LLM, dictionary, regulator and safety-database traffic leaves through dedicated services, never directly from the UI or the database.

Key design decisions:

- **Start as a modular monolith with workers.** It's easier to validate and operate than many microservices. Strict module boundaries let busy parts, such as the AI workers, be split out later.
- **The workflow engine owns case state and clocks.** Deadlines survive crashes and deploys, and every step is retried safely.
- **A single AI gateway.** It's the one place for redaction checks, model routing, budgets, logging and provider fallback.
- **Deploy per region.** Each tenant's data, processing and LLM calls stay in its residency region (EU, US or India).

Case lifecycle and AI design

Each case runs as a durable workflow. AI steps are separate activities that can be retried; every regulated decision waits for a signed human action.

Lifecycle

1. **Received:** the source is stored in immutable storage, checked against the idempotency key, and acknowledged to the sender.
2. **Pre-processing:** OCR, language detection, splitting a file that holds several reports, and personal-data redaction.
3. **AI processing:** extraction, validity checks, seriousness, duplicates, coding suggestions, follow-up questions and a narrative draft.
4. **Triage:** the case is valid, invalid (held for follow-up), or a duplicate candidate (sent for a merge decision). The regulatory clock starts here.
5. **Data entry review:** a case processor confirms or corrects every field and signs.
6. **Quality and medical review:** a second person checks serious cases, plus a sampled share of non-serious cases. A physician assesses whether the drug likely caused the event.
7. **Approved:** the case is locked and given a new version number.
8. **Submitted:** E2B(R3) is generated, validated and sent; acknowledgments are tracked and failures re-queued.
9. **Follow-up:** new information creates a new case version, re-runs the affected AI steps and restarts the relevant clocks.

AI pipeline design

- **Separate deterministic logic from AI.** Rules handle validity, dates, units, clocks and E2B business rules. AI only reads text and proposes values, always with an evidence span and a confidence score.
- **Two signals for seriousness.** The LLM and a rule engine assess independently. If they disagree, the case is treated as serious, because missing a serious case costs far more than a false alarm.
- **Coding as retrieval, not generation.** Search the customer's licensed MedDRA and WHODrug versions for candidate terms, then let the LLM pick from that list. It can never invent a code.
- **Calibrated confidence.** Raw model confidence is adjusted against reviewer corrections so that 0.9 really means about 90% correct. Low-confidence fields are highlighted for review.
- **Model routing through an AI gateway.** A smaller model handles simple steps such as language detection and splitting; a stronger model handles extraction and seriousness. Every call is logged, size-limited and time-limited, and a fallback model is configured.

Automation levels (set per tenant and per case type)

Level	What AI does	Human role	Default
L0 Assist	Suggests values in the side panel	Enters all data	For validation runs
L1 Pre-fill	Fills the form with evidence	Confirms every field and signs	Default for all cases
L2 Straight-through	Completes non-serious, valid, high-confidence cases	Samples a share of cases for QC and signs in bulk	Opt-in, only after proven accuracy

Serious cases never go beyond L1.

Model risk management

- **One versioned AI configuration.** Prompts, model versions, retrieval settings and thresholds are released together, as one versioned unit, under change control.

- **Release gate.** Every new configuration runs against a locked, labelled validation set. Release needs seriousness recall of at least 0.98 and no field-level drop beyond agreed limits. The results are stored as validation evidence.
- **Shadow mode before switching.** The new configuration runs alongside production on live cases without affecting them. Reviewers' choices are compared with both versions.
- **Production monitoring.**
  - Reviewer edit rate per field.
  - Seriousness overrides.
  - Duplicate decisions.
  - Confidence drift.
  - Cost per case.

  Alerts fire when a metric crosses its threshold.
- **Instant rollback** to the previous approved configuration. Every case records which configuration processed it.

Data, multi-tenancy and integrations

The case data model follows the E2B(R3) structure from the start, so submission is a mapping step rather than a redesign.

Data model principles

- **E2B-aligned case model:** patient, reporter, drugs, reactions, tests, narrative and causality, mirroring the main E2B(R3) sections.
- **Every case is versioned.** A version is never edited after approval; follow-ups create a new version. Old versions stay readable as they were.
- **Field-level history:** each field keeps its value, where it came from (AI, human or import), the evidence span, confidence, and who confirmed it.
- **Tamper-evident audit ledger:** an append-only table in which each entry includes a hash of the previous one. A nightly job verifies the chain and archives it to write-once (WORM) storage.
- **Transactional outbox:** a state change and its outgoing event are written in the same database transaction, so other services never miss or duplicate an event.

Multi-tenancy

Model	Used for	How
Pooled (default)	Small and mid-size tenants	Shared database; tenant_id on every row enforced by PostgreSQL row-level security; separate encryption keys per tenant
Dedicated	Large pharma, strict residency or audit needs	Own database and storage in the same deployment, or a fully separate deployment in a chosen region

Tenant context comes from the signed token. It is never accepted from request parameters. Automated tests confirm that cross-tenant reads fail.

Integrations

Integration	Direction	Method	Notes
Email inbox	In	Microsoft Graph or IMAP polling	Attachments go to immutable storage; delivery receipts are logged
Web and call-centre forms	In	REST API with idempotency keys	Embeddable form for customer websites
Partner and licensee cases	In and out	E2B(R3) XML over AS2 or SFTP	Schema and business-rule validation on import
Literature	In	PubMed API on a schedule	AI screens articles for possible cases; a human confirms
Regulators	Out	E2B(R3) through FDA ESG and EMA EudraVigilance gateways; national authorities via configurable connectors	Acknowledgments are parsed and failed submissions re-queued
Customer safety databases	Out	E2B(R3) export or vendor APIs (e.g. Oracle Argus, ArisGlobal LifeSphere)	Many customers keep their own system of record

Integration	Direction	Method	Notes
Medical dictionaries	In	Customer uploads licensed MedDRA and WHODrug releases	Several versions held side by side; version changes handled by a re-coding job
Identity	In	OIDC or SAML single sign-on with each customer's identity provider; SCIM for user provisioning	No local passwords in production

Security, compliance and validation

Compliance is designed into the platform as product features, so customers can validate it with evidence the vendor supplies, not with custom work for every customer.

Part 11 and Annex 11 controls as features

Requirement	Product feature
Secure, time-stamped audit trail	Hash-chained ledger of who, what, when, old value, new value and reason; cannot be edited by any user, including admins
Electronic signatures	Re-authentication at signing; the record shows signer name, date and time, and meaning (e.g. "Reviewed", "Medically approved"); signatures are permanently linked to that case version
Access control	Role-based plus data-scope rules (tenant, product, region); periodic access reviews; automatic session timeout
Accurate copies for inspection	Export of any case version, with its audit trail, as PDF and E2B XML
System validation	Validation package per release (see below)
Operational checks	Workflow enforces step order, e.g. no submission before approval

Validation approach (GAMP 5, risk-based)

- **Classify by risk.** Functions that affect patient safety or regulatory submissions (seriousness, clocks, E2B generation, signatures) get full scripted testing. Lower-risk features get unscripted or automated testing.
- **Ship a validation pack with each release:**
  - Requirements traced to tests.
  - Automated test evidence.
  - AI validation results.
  - Release notes with an impact assessment.
- **Locked environments.** A dedicated validation environment runs the exact release candidate. Customers can run their acceptance tests there before production.
- **Change control.** Releases follow a published calendar. AI configuration changes go through the same process as code.

Privacy

- **Redact before the LLM.** Direct identifiers (names, contact details, addresses, ID numbers) are replaced with tokens before any model call. The mapping is encrypted and only the case service can reverse it.
- **Data residency.** Each tenant is pinned to a region (EU, US or India). Storage, processing and LLM endpoints stay in that region.
- **Data subject rights** (GDPR and the DPDP Act) are handled together with regulatory retention duties. Safety data usually must be kept, so requests are recorded and handled under documented rules.

Security architecture

- **Network:** zero trust between services, with mutual TLS. Databases, storage and LLM endpoints are reached only through private endpoints. A web application firewall sits in front of every public endpoint.
- **Encryption and secrets:** keys live in Key Vault, with customer-managed keys optional. Secrets are never stored in code or environment files.
- **Software supply chain:** signed container images and a software bill of materials. Dependency scanning and static analysis run in CI.
- **LLM provider terms:** contracts guarantee no data retention and no training on customer data.

LLM-specific threats

Threat	Control
Prompt injection in a report or attachment	Reports are wrapped and labelled as data; the extraction model has no tools; output is schema-validated; an injection test suite runs in CI
Hallucinated values	Every field must have an evidence span in the source; values without evidence are rejected; codes come only from dictionary search
Leakage between tenants	No shared prompts or caches containing tenant data; per-tenant keys; strict tenant isolation in the AI gateway
Silent model change by a provider	Pinned model versions; scheduled re-validation before any upgrade
Over-reliance by reviewers	Evidence shown beside every field; sampled QC; reviewer edit rate monitored

Reliability, scale and operations

The one failure the product must never have is a missed regulatory deadline. Reliability work starts from that.

Service level objectives

Objective	Target	Alert when
Platform availability	99.9% per month	Error budget burn rate exceeds 2× for 1 hour
Intake acknowledged	99.99% of reports stored within 60 seconds	Any report unacknowledged after 5 minutes
AI processing	p95 under 2 minutes	p95 over 5 minutes for 15 minutes
Clock safety	100% of serious cases visible in a queue with at least 48 hours to deadline	Any serious case under 72 hours and unassigned

Objective	Target	Alert when
Submission	99.5% of submissions acknowledged without technical error	Any technical rejection

Reliability patterns

- **Durable workflows.** Temporal holds each case's state, timers and retries, so a crash or deploy never loses work or resets a clock.
- **Idempotency everywhere.** Each report and submission has an idempotency key. Retries can never create a second case or a duplicate submission.
- **Graceful degradation.** If the LLM provider is down, cases fall back to a second provider or to manual entry (L0) with a banner. Intake and clocks keep running.
- **Backpressure.** Queue depth limits and per-tenant rate limits stop one tenant's spike from slowing others.
- **Disaster recovery.**
  - The database is zone-redundant, with point-in-time restore.
  - Storage is geo-redundant within the tenant's residency region.
  - Restore is tested every quarter against RPO 15 minutes and RTO 4 hours.

Observability

- **One trace per case.** OpenTelemetry traces follow a case from intake through the AI steps to submission, linked to its case id.
- **LLM telemetry.** For every model call, record prompt version, model, tokens, latency, cost, and the outcome of the reviewer's check. Store this redacted, in a self-hosted LLM tracing tool.
- **Business dashboards:**
  - Backlog and age.
  - Cases at risk of missing a deadline.
  - On-time submission rate.
  - Reviewer productivity.
  - AI edit rate per field.

Scale and cost

- **Scale settings.** AI workers scale on queue depth, and API pods on CPU and request rate. PostgreSQL is sized for writes, with read replicas for search and reports.
- **Cost controls:**
  - Route simple steps to small models.
  - Cache prompt prefixes.
  - Batch literature screening.
  - Track LLM cost per case against a budget for each tenant.
- **Unit cost.** Track cost per approved case, covering compute, LLM and storage. It is the number that sets the product's price.

Technology choices

The stack favours boring, well-supported components and your existing strengths: Python, FastAPI, Azure and React.

Layer	Choice	Why	Alternative considered
Backend services	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2	One language across APIs and AI; strong typing with Pydantic	.NET for core services: viable, but two stacks double hiring and tooling
Workflow orchestration	Temporal (Python SDK)	Durable state, timers for regulatory clocks, retries, workflow versioning, human-task waits	LangGraph alone: good for the AI steps but not for long-lived business workflows with deadlines
AI step logic	Plain Python steps inside Temporal activities; LangGraph only where a step needs branching	Keeps each AI step small, testable and replaceable	A full agent framework for everything: harder to validate

Layer	Choice	Why	Alternative considered
LLM access	Internal AI gateway service in front of Azure OpenAI and a second provider	Routing, redaction checks, budgets, logging and fallback in one place	Direct SDK calls from each service: no central control
Database	Azure Database for PostgreSQL (Flexible Server), with pgvector and full-text search	Relational case data, row-level security, vectors for duplicates; one engine to operate	Dedicated vector store and OpenSearch: add later if search outgrows Postgres
Events	Azure Service Bus with a transactional outbox	Reliable integration events, dead-letter queues	Kafka: more power than needed at this volume
Documents	Azure Blob Storage with immutability (WORM) policies	Source documents and audit archives that cannot be changed	—
OCR	Azure AI Document Intelligence	Handles scans, forms and tables	Tesseract: kept for local development
Frontend	React, TypeScript, Vite, TanStack Query, an accessible component library	Fast, typed review UI	Next.js: not needed for an authenticated single-page app
Identity	Microsoft Entra ID for staff; customer IdPs through OIDC or SAML; SCIM	Enterprise single sign-on is a buying requirement	Local accounts: development only (Keycloak)
Compute and infra	AKS, Helm, Terraform; Azure Front Door with WAF; Key Vault	Portable, repeatable environments per region	Container Apps: simpler, but less control for dedicated tenants

Layer	Choice	Why	Alternative considered
Observability	OpenTelemetry, Azure Monitor or Grafana stack, self-hosted Langfuse	One trace from request to model call	—
CI/CD	GitHub Actions; environments dev → test → validation → production	Automated evidence for each release	—

Build prompt

Paste this into a coding agent (for example Claude Code) at the root of an empty repository. It builds in 11 phases, each with acceptance criteria. Review and test every phase before telling the agent to continue.

```
# Build PharmaSafe: a product-grade, AI-assisted adverse event case
processing platform

## Context
PharmaSafe is a multi-tenant SaaS product for pharmacovigilance. It receives
adverse event (AE) reports from many channels, uses AI to extract and pre-
process them into structured safety cases, routes them through human review
with electronic signatures, and produces regulator-ready E2B(R3) submissions.
It must be built so that customers can validate it under GxP, 21 CFR Part 11
and EU Annex 11.

Core principle: AI proposes, humans decide. Deterministic code handles
validity rules, dates, regulatory clocks and E2B business rules. AI only
reads text and proposes values, each with an evidence span and a confidence
score. No AI output becomes part of an approved case without a signed human
action.

Use only synthetic or public data. MedDRA and WHODrug are licensed: build a
dictionary adapter and use a small synthetic test dictionary in development.
Never commit licensed content.

## Architecture
```

```
Build a modular monolith plus workers, not dozens of microservices. Keep
module boundaries strict (no cross-module database access; communicate
through interfaces or events) so modules can be split out later.

Deployable units:
- api: FastAPI app with modules: identity, tenancy, cases, intake, review,
admin, reporting, audit
- workflow-worker: Temporal workers for the case workflow and regulatory
clocks
- ai-worker: Temporal activities for the AI pipeline
- ai-gateway: internal service that is the only path to LLM providers
- submission-worker: E2B(R3) generation, validation, gateway connectors,
acknowledgment handling
- review-ui: React SPA

Stack: Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2, Alembic, PostgreSQL
16 with pgvector and full-text search, Temporal (Python SDK), Azure Service
Bus with a transactional outbox (a local emulator or a Postgres-backed queue
in development), Azure Blob Storage with immutability (Azurite locally),
Azure AI Document Intelligence behind an OCR interface (Tesseract locally),
React + TypeScript + Vite + TanStack Query, OpenTelemetry, self-hosted
Langfuse, Keycloak locally for OIDC, Terraform + Helm for Azure (AKS, Front
Door with WAF, Key Vault, Entra ID).

Repository layout:
- apps/api, apps/workflow_worker, apps/ai_worker, apps/ai_gateway,
apps/submission_worker, apps/review_ui
- packages/domain (E2B-aligned case model, rules), packages/platform (auth,
tenancy, audit, telemetry, outbox)
- eval/ (datasets, harness, reports), tools/synthetic_data/
- infra/terraform, infra/helm, deploy/docker-compose.yml
- docs/adr (architecture decision records), docs/validation (traceability
matrix, test evidence), docs/runbooks

## Domain rules
1. Case validity = 4 minimum criteria: identifiable patient, identifiable
reporter, at least one suspect drug, at least one adverse reaction.
2. Seriousness criteria (ICH E2A): death, life-threatening, hospitalisation
(initial or prolonged), persistent or significant disability, congenital
anomaly, other medically important event. Each is a separate field with
evidence.
3. Day 0 = the date the company first had all 4 minimum criteria. Due dates
come from versioned, configurable rule sets per regulator (FDA, EMA, CDSCO),
```

```
seeded with example rules that are clearly marked as configurable.
4. Case model mirrors the main E2B(R3) sections. Approved case versions are
immutable; follow-up information creates a new version.
5. Every field stores: value, source (ai / human / import), evidence span,
confidence, confirmed_by, confirmed_at.

## Cross-cutting requirements (apply in every phase)
- Multi-tenancy: tenant_id on every tenant table, enforced with PostgreSQL
row-level security; tenant comes only from the verified token. Tests must
prove cross-tenant access fails.
- Audit: an append-only, hash-chained audit ledger (who, what, when, old
value, new value, reason, case version). No update or delete permissions on
it for the application role. A job verifies the chain.
- Electronic signatures: re-authentication at signing; record signer,
timestamp and meaning; bind the signature to a specific case version.
- Idempotency keys on every intake and submission endpoint.
- Structured JSON logs and OpenTelemetry traces with case_id and tenant_id on
every span. Never log unredacted personal data.
- Typed code (mypy strict on packages/domain), ruff, pytest; minimum 85%
coverage on packages/domain.
- Each significant decision gets a short ADR in docs/adr.
- No secrets in the repository; .env.example only.

## Phases
Stop after each phase. Report what was built, how to run it, test results,
and any deviations. Wait for my go-ahead.

Phase 0 - Foundations
Monorepo, tooling, docker-compose with Postgres, Temporal, Azurite, Keycloak,
Langfuse and a local mail server (GreenMail). CI on GitHub Actions: lint,
type check, tests, dependency and static security scanning, SBOM generation.
Accept: `make up` starts everything; CI passes on an empty skeleton.

Phase 1 - Identity, tenancy, audit
OIDC login via Keycloak; roles: intake_processor, case_processor,
quality_reviewer, medical_reviewer, tenant_admin, platform_admin; data scopes
by product and region; row-level security; audit ledger; e-signature service.
Accept: tests prove cross-tenant isolation, audit immutability, chain
verification and signature binding.

Phase 2 - Case domain
E2B-aligned case model, versioning, field provenance, validity rules, date
and unit checks, regulatory clock rule engine, case APIs, transactional
```

```
outbox.
Accept: property-based tests for validity and clock rules; a past case
version can be rebuilt exactly.

Phase 3 - Intake
REST intake with idempotency, web form, email polling (GreenMail locally),
file upload, immutable source storage, OCR interface, language detection,
multi-report splitting.
Accept: the same email processed twice creates one case; every source
document is retrievable byte-for-byte.

Phase 4 - Workflow
Temporal case workflow: received -> pre-processing -> AI processing -> triage
-> data entry review -> quality/medical review -> approved -> submitted ->
follow-up. Human tasks, assignment, four-eyes rule, escalation timers before
deadlines, workflow versioning.
Accept: killing workers mid-case loses nothing and does not reset clocks;
tests use Temporal's time-skipping test environment for deadline escalations.

Phase 5 - AI gateway and AI pipeline
ai-gateway: provider adapters (Azure OpenAI plus one other), a MockLLM for
tests, model routing per step, timeouts, max tokens, retries with backoff,
fallback provider, per-tenant budgets, redacted request logging to Langfuse,
pinned model versions.
Pipeline activities: personal-data redaction (reversible tokens, encrypted
mapping), extraction with structured output and evidence spans (reject any
value without valid evidence), dual-signal seriousness (LLM plus rule engine;
disagreement = serious and flagged), duplicate detection (blocking keys plus
pgvector similarity, top 3 with reasons), dictionary coding (retrieve
candidates from the dictionary adapter, LLM picks only from candidates),
follow-up questions, narrative draft from confirmed fields only.
AI configuration (prompts, models, thresholds, retrieval settings) is a
versioned, signed bundle; every case records the bundle version used.
Automation levels L0/L1/L2 per tenant; serious cases are capped at L1.
Treat report text as untrusted: delimit it, give the extraction model no
tools, validate all output against schemas.
Accept: full pipeline runs on MockLLM in CI; an injection test suite passes;
a provider outage falls back to the second provider, then to L0 manual mode.

Phase 6 - Review UI
Queues (sorted by deadline risk), case workspace (source document with
evidence highlighting next to the editable E2B-structured form, confidence
badges, seriousness panel showing LLM vs rules, duplicate comparison and
```

```
merge, coding picker, follow-up questions, narrative), e-signature dialog,
case version history and audit viewer, tenant admin screens, accessibility to
WCAG 2.1 AA.
Accept: Playwright tests for intake-to-approval, duplicate merge and follow-
up versioning.

Phase 7 - Submission and exchange
E2B(R3) XML generation from approved versions, validation against the ICH
schemas (obtain them from the ICH website; check their terms before
committing) and configurable regional business rules, gateway connector
interface with a mock FDA ESG / EudraVigilance gateway, acknowledgment
parsing, re-queue on failure, E2B import for partner cases, export for
customer safety databases.
Accept: round-trip tests (import -> case -> export) preserve data; a rejected
acknowledgment creates a visible work item.

Phase 8 - Evaluation and model risk management
Synthetic data generator (300+ labelled reports: consumer and HCP styles,
missing data, misspellings, brand vs generic names, multi-report files, non-
English text, near-duplicates, serious cases, injection attempts). Eval
harness: field-level precision/recall/F1, validity accuracy, seriousness
recall, duplicate precision/recall, coding top-1 and top-3 accuracy,
injection resistance, latency and cost per case. Release gate in CI: block if
seriousness recall < 0.98 or any field F1 drops > 2 points from the approved
baseline. Shadow mode: run a candidate bundle alongside production and
compare against reviewer decisions. Monitoring: reviewer edit rate per field,
seriousness overrides, confidence drift, cost per case, with alert
thresholds.
Accept: an eval report is generated as markdown and JSON and attached to the
release; a regressed prompt fails the gate.

Phase 9 - Reliability, observability, security hardening
SLO definitions and dashboards (availability, intake acknowledgment, AI p95,
deadline risk, submission success), alerts, per-tenant rate limits and
backpressure, load test (k6 or Locust) at 100,000 cases per month with peak
bursts, chaos tests (kill workers, database failover, LLM outage), threat
model document, dependency pinning and image signing.
Accept: load and chaos test reports in docs/runbooks; no lost or duplicated
cases under failure.

Phase 10 - Infrastructure, operations and validation pack
Terraform for Azure (per-region deployment, AKS, PostgreSQL Flexible Server
zone-redundant, Service Bus, Blob with immutability, Key Vault with customer-
```

```
managed key option, Front Door with WAF, private endpoints), Helm charts,
environments dev/test/validation/prod, zero-downtime deploys, backup and
restore runbook tested against RPO 15 min / RTO 4 h. Generate a validation
pack per release: requirements-to-test traceability matrix, automated test
evidence, AI eval report, risk assessment, release notes.
Accept: `terraform plan` succeeds for a new region; the validation pack is
produced by CI.

## Working rules
- Build the smallest correct version of each phase; no speculative features.
- Explain non-obvious decisions in code docstrings and ADRs.
- Ask before adding major dependencies or changing the architecture.
- If something in this prompt is ambiguous or conflicts, stop and ask.
```

What the build cannot cover, and open questions

The prompt produces the software foundation. A coding agent cannot supply the items below; they need people, contracts and time.

Outside the code

Item	Why it matters	Who owns it
MedDRA and WHODrug licences	Coding with real dictionaries is mandatory for submissions	Each customer licenses them; the vendor needs a development licence
Regulator gateway onboarding	Test and production accounts with FDA ESG and EudraVigilance take formal registration and testing	Customer, supported by the vendor
Formal validation	Validation is completed by each customer, in its own quality system, using the vendor's pack	Customer QA, with vendor QA support
Quality management system	SOPs, training records, change control, CAPA, supplier audits	Vendor QA function
Real-world AI validation	Accuracy on thousands of real, de-identified cases with drug safety experts labelling them	A design-partner customer

Item	Why it matters	Who owns it
Security certifications	ISO 27001 and SOC 2 are usually required in enterprise buying	Vendor security team
Domain experts	A drug safety physician and a regulatory specialist must review rules, workflows and the UI	Hired or advisory

Open questions

- Which regulators come first? The design assumes FDA and EMA, then CDSCO.
- Should PharmaSafe be a customer's system of record, or feed an existing safety database? This changes the scope of the UI and reporting.
- Will any customer require on-premises deployment, or only dedicated cloud deployments?
- Which LLM providers are approved in each customer's region and contract?
- What straight-through (L2) rate do customers actually want, and what QC sampling share will their quality teams accept?

For your portfolio

For a solo build, phases 0–6 and 8 are realistic over a few months. They make a strong, honest demo: "designed for product-grade compliance and scale, implemented through review UI and AI evaluation." Phases 7, 9 and 10 are good topics for system design interviews even if you only partly build them.